

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Code:	CSA0309	Course Title:	Data Structures
Assessed:	CO2 – Manipulation (BL3, BL4)	Assessment:	Assessment Tool 1 – Analytical Problem Solving
Weightage:	30% of CIA	Total Marks:	100
Statement:	Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.		
Student Name:	K.Nilavan	Reg. No.:	192511028
Year / Batch:	2025 – 2029	Date:	17/07/2026

Declaration of Conduct

Nilavan (192511028) (Name / Reg No)

I declare that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: K. Nilavan

Instructions

This test contains 80 Analytical problem-solving questions. Total: 100 Marks.

When a student answer using an Analytical problem-solving, in evaluation the answer should be with following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.

Each student should write 2 questions. No negative marking. Unanswered questions carry zero marks.

No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic		Focus	Q Nos	Marks
Data Structure, Search Data Structure		List Operations, Binary Search	1	50
Data Structure, Queue Data Structures		Stack Notations, Priority Queue	2	50
GRAND TOTAL:			100 Marks	

Analytical Problem Solving

1)

Two algorithms solve the same problem. Algorithm A uses recursion, while Algorithm B uses explicit stack. Compare both.

* Algorithms can solve the same problem in different ways. The common approaches include recursion & explicit stack. Both follow the Last In, First Out (LIFO) principle, but they differ in another way.

→ Algorithm A: Recursion

• A recursive algorithm solves a problem by breaking it into smaller subproblems and calling itself repeatedly until a base condition is reached.

↳ Characteristics:

- * Uses the system call stack.
- * Every function call is stored in memory.

↳ Advantages:

* Requires less code for problems like tree traversal, factorial and Fibonacci

→ Algorithm B: Explicit Stack.

• In this approach, a stack data structure is created by the programmer instead of relying on recursive function calls.

⇒ characteristics:

- * Uses user-defined stack
- * Programmer controls push & pop operations.

⇒ Advantages:

- * Better control over memory usage.
- * Suitable for large inputs.

Feature	Recursion	Explicit Stack
Memory Usage	uses system call stack	uses user-defined stack.
Execution Flow	Automatic function calls	Manual push & pop operations.
Control	Managed by the system.	Managed by the programmer
Risk of stack overflow	High for deep recursion	Very low
code length	shorter and simpler	longer but more flexible
Performance	Extra function call overhead	Faster for large problems.

Conclusion:

* Both recursion and explicit stack follow the LIFO principle. Recursion is simple & suitable for small problems & stack provides better control, improved memory management.

A stack has a capacity of 5 elements. Explain what happens if the following operations are executed: push(A), push(B), push(C), push(D), push(E), push(F). Identify errors & suggest 2 solutions.

→ Problem Statement:

* A stack has maximum capacity of 5 elements.

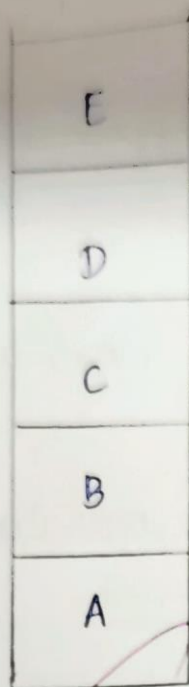
* The following operations are performed:

- push(A)
- push(B)
- push(C)
- push(D)
- push(E)
- push(F)

Execution using stack operations:-

Operation	Stack Contents	Result
Push(A)	A	Success
Push(B)	AB	Success
Push(C)	ABC	Success
push(D)	AB CD	Success
Push(E)	ABCDE	Success (Stack Full).
Push(F)	ABCDE	overflow

Stack Diagram:



* When push(F) is executed, no free location is available.

Error Identified:

* The error is called stack overflow.

Definition:

• Stack overflow occurs when a push() operation is performed on a stack that has already reached its maximum capacity.

• For a stack of size n , after inserting A, B, C, D, E, the TOP pointer points to the last position.

Data Structure principle:

- For an array implementation:

```
if (TOP == MAX-1)
    Stack overflow
else
    Push(Element)
```

Here,

- MAX = 5
- TOP = 4 after inserting E.
- Attempting another push results in overflow.

Possible solutions:

Solution - 1: Overflow checking:

* Before every push(), check whether the stack is full.

```
if (TOP == MAX-1)
    Display "Stack overflow"
```

else

```
Push(Element)
```

* This prevents invalid insertion.

Solution - 2: Dynamic Stack implementation.

* Implement the stack using linked list instead of an array.

Advantages:

- Memory is allocated dynamically.
- Stack size grows as required.
- Overflow occurs only when system memory is exhausted.

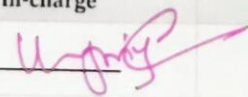
Conclusion:

* Using concepts like Stack ADT, LIFO, call stack & TOP pointer, array implementation & linked-list implementation makes more efficient.

38	192511310	VAIBHAVA LAKSHMI K	- Two algorithms solve the same problem. Algorithm A uses recursion, while Algorithm B uses an explicit stack. Compare both approaches in terms of memory usage, execution flow, and risk of stack overflow. - A stack has a capacity of 5 elements. Explain what happens if the following operations are executed: Push(A), Push(B), Push(C), Push(D), Push(E), Push(F). Identify the error and suggest two possible solutions.
39	192524458	VASANTHAN S	- Compare the performance of a linear queue and a circular queue when 100 enqueue and dequeue operations are performed alternately. - A queue implemented using an array report "Queue Full" even though some positions are empty. - Analyze why this occurs. - Explain how a circular queue solves this problem.
40	192524083	YARRAPPAREDDYG ARI SAI GOPAL REDDY	- Consider the singly linked list: 10 → 20 → 30 → 40 → 50 Delete the node containing 30. Explain how the links change after deletion. - Given two sorted linked lists: L1: 1 → 3 → 5 → 7 L2: 2 → 4 → 6 → 8

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20	Demonstrates complete and precise understanding; identifies all constraints and edge cases	Shows clear understanding with minor gaps	Partial understanding; misses some constraints	Misinterprets problem or lacks clarity
Data Structure Selection	20	Selects optimal data structure with strong justification and comparison	Selects appropriate structure with reasonable justification	Selects somewhat suitable structure with weak reasoning	Incorrect or no data structure selection
Algorithm Design	20	Designs efficient, logically sound, and well-structured algorithm	Algorithm is correct with minor inefficiencies	Algorithm is partially correct or lacks clarity	Algorithm is incorrect or missing
Accuracy of Solution	30	Error-free, well-structured, optimized code/solution	Minor errors but overall functional	Multiple errors; partially working	Non-functional or missing solution
Clarity	10	Exceptionally clear, well-organized, uses diagrams effectively	Clear and organized with minor issues	Somewhat organized but lacks clarity	Poorly organized and unclear

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____